Questões 901-1000 — Recursão Avançada e Algoritmos Clássicos (Difícil)

Cem questões de lógica de programação envolvendo recursão avançada, backtracking, busca binária, ordenação, grafos básicos e programação dinâmica introdutória.

Questão 901

Dificuldade: Difícil
Implemente busca binária iterativa para encontrar a posição de um valor em um vetor ordenado de forma crescente.

Entrada: Vetor [2, 5, 8, 12, 16, 23, 38, 45], valor procurado 23 **Saída:** Posição 5 (índice baseado em zero)

Questão 902

Dificuldade: Difícil
Implemente busca binária recursiva que retorne -1 caso o valor não esteja presente no vetor ordenado.

Entrada: Vetor [1, 3, 5, 7, 9, 11], valor procurado 6 **Saída:** -1

Questão 903

Dificuldade: Difícil
Usando busca binária, encontre a primeira posição em que um valor poderia ser inserido em um vetor ordenado mantendo a ordem (posição de inserção).

Entrada: Vetor [1, 3, 3, 5, 8], valor a inserir 4 **Saída:** Posição 3

Questão 904

Dificuldade: Difícil
Implemente o algoritmo de ordenação por seleção (selection sort) do zero, mostrando o vetor resultante.

Entrada: Vetor [29, 10, 14, 37, 13] **Saída:** [10, 13, 14, 29, 37]

Questão 905

Dificuldade: Difícil
Implemente o algoritmo de ordenação por inserção (insertion sort) do zero.

Entrada: Vetor [12, 11, 13, 5, 6] **Saída:** [5, 6, 11, 12, 13]

Questão 906

Dificuldade: Difícil

Implemente o algoritmo bolha (bubble sort) contando quantas trocas foram realizadas até a ordenação completa.

Entrada: Vetor [5, 1, 4, 2, 8] **Saída:** Vetor ordenado [1, 2, 4, 5, 8], número de trocas: 4

Questão 907

Dificuldade: Difícil
Implemente o merge sort de forma recursiva, descrevendo a divisão do vetor e a etapa de intercalação (merge).

Entrada: Vetor [38, 27, 43, 3, 9, 82, 10] **Saída:** [3, 9, 10, 27, 38, 43, 82]

Questão 908

Dificuldade: Difícil
Implemente a função de intercalação (merge) usada no merge sort, que combina dois subvetores já ordenados em um único vetor ordenado.

Entrada: Subvetor A [1, 5, 9], Subvetor B [2, 3, 8] **Saída:** [1, 2, 3, 5, 8, 9]

Questão 909

Dificuldade: Difícil
Implemente o quicksort recursivo usando o último elemento como pivô, mostrando o vetor final ordenado.

Entrada: Vetor [10, 7, 8, 9, 1, 5] **Saída:** [1, 5, 7, 8, 9, 10]

Questão 910

Dificuldade: Difícil
Implemente a função de particionamento (partition) do quicksort, retornando a posição final do pivô após o particionamento.

Entrada: Vetor [8, 3, 1, 7, 0, 10, 2], pivô = último elemento (2) **Saída:** Posição final do pivô: 2, vetor particionado: [1, 0, 2, 7, 8, 10, 3] (uma possível saída válida)

Questão 911

Dificuldade: Difícil
Usando o algoritmo de particionamento do quicksort (quickselect), encontre o k-ésimo menor elemento de um vetor sem ordená-lo completamente.

Entrada: Vetor [7, 10, 4, 3, 20, 15], k = 3 **Saída:** 7

Questão 912

Dificuldade: Difícil

Resolva a Torre de Hanói para N discos usando recursão, retornando o número total de movimentos necessários.

Entrada: Número de discos = 4 **Saída:** 15 movimentos

Questão 913

Dificuldade: Difícil
Resolva a Torre de Hanói para 3 discos e liste a sequência de movimentos no formato "mover disco da torre X para a torre Y".

Entrada: 3 discos, torres A (origem), B (auxiliar), C (destino) **Saída:** mover disco 1 de A para C; mover disco 2 de A para B; mover disco 1 de C para B; mover disco 3 de A para C; mover disco 1 de B para A; mover disco 2 de B para C; mover disco 1 de A para C

Questão 914

Dificuldade: Difícil
Usando backtracking, gere todas as permutações possíveis de um conjunto de elementos distintos.

Entrada: Conjunto [1, 2, 3] **Saída:** [1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]

Questão 915

Dificuldade: Difícil
Usando backtracking, gere todos os subconjuntos possíveis (conjunto das partes) de um conjunto de elementos distintos.

Entrada: Conjunto [1, 2, 3] **Saída:** {}, {1}, {2}, {3}, {1,2}, {1,3}, {2,3}, {1,2,3}

Questão 916

Dificuldade: Difícil
Resolva o problema das N-rainhas para um tabuleiro 4x4 usando backtracking, retornando o número de soluções distintas.

Entrada: Tabuleiro 4x4 **Saída:** 2 soluções

Questão 917

Dificuldade: Difícil
Resolva o problema das N-rainhas para um tabuleiro 4x4 e apresente uma das soluções válidas indicando a coluna ocupada em cada linha.

Entrada: Tabuleiro 4x4 **Saída:** Linha 1: coluna 2; Linha 2: coluna 4; Linha 3: coluna 1; Linha 4: coluna 3

Questão 918

Dificuldade: Difícil
Usando backtracking, determine se existe um subconjunto de um conjunto de números cuja soma seja exatamente igual a um valor alvo.

Entrada: Conjunto [3, 34, 4, 12, 5, 2], alvo = 9 **Saída:** Verdadeiro (subconjunto [4, 5])

Questão 919

Dificuldade: Difícil
Usando backtracking, liste todos os subconjuntos de um conjunto de números cuja soma seja igual a um valor alvo.

Entrada: Conjunto [2, 3, 5, 6, 8], alvo = 8 **Saída:** [3, 5] e [2, 6] e [8]

Questão 920

Dificuldade: Difícil
Calcule o N-ésimo número de Fibonacci utilizando recursão com memoização (programação dinâmica top-down).

Entrada: N = 30 **Saída:** 832040

Questão 921

Dificuldade: Difícil
Calcule o N-ésimo número de Fibonacci utilizando programação dinâmica bottom-up com uma tabela, sem usar recursão.

Entrada: N = 10 **Saída:** 55

Questão 922

Dificuldade: Difícil
Resolva o problema da mochila 0/1 simplificado usando programação dinâmica: dado um conjunto de itens com peso e valor, e uma capacidade máxima, encontre o valor máximo alcançável.

Entrada: Itens (peso, valor): (1,1), (3,4), (4,5), (5,7), capacidade = 7 **Saída:** Valor máximo = 9 (itens de peso 3 e 4)

Questão 923

Dificuldade: Difícil
Resolva o problema da mochila 0/1 e indique quais itens compõem a solução ótima, além do valor máximo.

Entrada: Itens (peso, valor): (2,3), (3,4), (4,5), (5,6), capacidade = 5 **Saída:** Valor máximo = 7 (itens de peso 2 e peso 3)

Questão 924

Dificuldade: Difícil
Encontre a maior subsequência comum (LCS) entre duas strings usando programação dinâmica, retornando o comprimento da subsequência.

Entrada: String A = "ABCBDAB", String B = "BDCABA" **Saída:** Comprimento 4 (exemplo: "BCBA")

Questão 925

Dificuldade: Difícil
Encontre a maior subsequência comum (LCS) entre duas strings usando programação dinâmica e retorne a própria subsequência encontrada.

Entrada: String A = "AGGTAB", String B = "GXTXAYB" **Saída:** "GTAB"

Questão 926

Dificuldade: Difícil
Usando programação dinâmica, calcule o número de caminhos distintos para ir do canto superior esquerdo ao canto inferior direito de uma grade, movendo-se apenas para a direita ou para baixo.

Entrada: Grade de 3 linhas por 3 colunas **Saída:** 6 caminhos

Questão 927

Dificuldade: Difícil
Usando programação dinâmica, calcule o número de caminhos distintos em uma grade com obstáculos, movendo-se apenas para a direita ou para baixo, onde células com obstáculo não podem ser visitadas.

Entrada: Grade 3x3 com obstáculo na posição linha 2, coluna 2 **Saída:** 2 caminhos

Questão 928

Dificuldade: Difícil
Usando programação dinâmica, encontre o número mínimo de moedas necessárias para formar um valor de troco, dado um conjunto de moedas disponíveis em quantidade ilimitada.

Entrada: Moedas disponíveis [1, 3, 4], valor alvo = 6 **Saída:** 2 moedas (3 + 3)

Questão 929

Dificuldade: Difícil
Usando programação dinâmica, determine de quantas formas diferentes é possível formar um valor de troco usando um conjunto de moedas disponíveis em quantidade ilimitada.

Entrada: Moedas disponíveis [1, 2, 5], valor alvo = 5 **Saída:** 4 formas diferentes

Questão 930

Dificuldade: Difícil
Implemente busca em largura (BFS) em um grafo representado por lista de adjacência, retornando a ordem de visita dos vértices a partir de um vértice inicial.

Entrada: Grafo: A-B, A-C, B-D, C-D, D-E; vértice inicial A **Saída:** Ordem de visita: A, B, C, D, E

Questão 931

Dificuldade: Difícil
Implemente busca em profundidade (DFS) em um grafo representado por lista de adjacência, retornando a ordem de visita dos vértices a partir de um vértice inicial.

Entrada: Grafo: A-B, A-C, B-D, C-E; vértice inicial A **Saída:** Ordem de visita: A, B, D, C, E

Questão 932

Dificuldade: Difícil
Usando DFS, detecte se existe um ciclo em um grafo não direcionado representado por lista de adjacência.

Entrada: Grafo: A-B, B-C, C-A, C-D **Saída:** Verdadeiro, existe ciclo (A-B-C-A)

Questão 933

Dificuldade: Difícil
Usando DFS com controle de estados de visita, detecte se existe um ciclo em um grafo direcionado.

Entrada: Grafo direcionado: A->B, B->C, C->A **Saída:** Verdadeiro, existe ciclo

Questão 934

Dificuldade: Difícil
Usando BFS ou DFS, conte quantos componentes conectados existem em um grafo não direcionado.

Entrada: Vértices A, B, C, D, E, F; arestas A-B, B-C, D-E **Saída:** 3 componentes conectados: {A,B,C}, {D,E}, {F}

Questão 935

Dificuldade: Difícil
Usando BFS, encontre o caminho mais curto (em número de arestas) entre dois vértices de um grafo não ponderado.

Entrada: Grafo: A-B, B-C, A-D, D-C, C-E; origem A, destino E **Saída:** Caminho A-D-C-E ou A-B-C-E, com 3 arestas

Questão 936

Dificuldade: Difícil
Implemente uma coloração simples de grafo: usando o mínimo de cores possível dado um limite fixo, atribua cores aos vértices de forma que vértices adjacentes nunca tenham a mesma cor, usando backtracking.

Entrada: Grafo: A-B, B-C, C-A, número máximo de cores = 3 **Saída:** A=cor1, B=cor2, C=cor3 (solução válida)

Questão 937

Dificuldade: Difícil
Usando backtracking, gere todas as combinações válidas de parênteses balanceados para um dado número de pares.

Entrada: Número de pares = 3 **Saída:** "((()))", "(()())", "(())()", "()(())", "()()()"

Questão 938

Dificuldade: Difícil
Calcule o fatorial de um número usando recursão de cauda (tail recursion), explicitando o acumulador utilizado.

Entrada: N = 6 **Saída:** 720

Questão 939

Dificuldade: Difícil
Usando recursão, calcule o máximo divisor comum (MDC) entre dois números pelo algoritmo de Euclides.

Entrada: 48 e 18 **Saída:** 6

Questão 940

Dificuldade: Difícil
Usando recursão, inverta uma string sem utilizar laços explícitos (for ou while).

Entrada: "programacao" **Saída:** "oacamargorp"

Questão 941

Dificuldade: Difícil
Usando recursão, verifique se uma string é um palíndromo, ignorando espaços e diferenças de maiúsculas/minúsculas.

Entrada: "A base do teto desaba" **Saída:** Verdadeiro

Questão 942

Dificuldade: Difícil
Usando recursão com backtracking, encontre todas as formas de dividir um número inteiro positivo em soma de inteiros positivos menores ou iguais a ele (partições do número).

Entrada: Número = 4 **Saída:** 4; 3+1; 2+2; 2+1+1; 1+1+1+1

Questão 943

Dificuldade: Difícil
Usando recursão, calcule o número de formas de subir uma escada de N degraus, podendo subir 1 ou 2 degraus por vez, aplicando memoização para eficiência.

Entrada: N = 5 degraus **Saída:** 8 formas

Questão 944

Dificuldade: Difícil
Usando programação dinâmica, calcule o número de formas de subir uma escada de N degraus podendo subir 1, 2 ou 3 degraus por vez.

Entrada: N = 4 degraus **Saída:** 7 formas

Questão 945

Dificuldade: Difícil
Usando recursão com backtracking, resolva um labirinto representado por uma grade, encontrando se existe caminho da entrada até a saída movendo-se apenas para cima, baixo, esquerda ou direita, sem passar por células bloqueadas.

Entrada: Grade 3x3 com bloqueio na posição central; entrada canto superior esquerdo; saída canto inferior direito **Saída:** Verdadeiro, existe caminho contornando o bloqueio

Questão 946

Dificuldade: Difícil
Usando backtracking, resolva um Sudoku parcial simplificado em um tabuleiro 4x4 (valores de 1 a 4), preenchendo as células vazias respeitando linhas, colunas e blocos 2x2.

Entrada: Tabuleiro 4x4 com algumas células vazias (representadas por 0) **Saída:** Tabuleiro completo respeitando as regras de linha, coluna e bloco

Questão 947

Dificuldade: Difícil
Implemente busca binária para encontrar a raiz quadrada inteira de um número (maior inteiro cujo quadrado não ultrapasse o valor).

Entrada: Número = 50 **Saída:** 7 (pois $7^2 = 49 \leq 50$ e $8^2 = 64 > 50$)

Questão 948

Dificuldade: Difícil
Usando busca binária, encontre o elemento pivô (menor elemento) em um vetor ordenado que foi rotacionado.

Entrada: Vetor rotacionado [15, 18, 2, 3, 6, 12] **Saída:** Menor elemento é 2, na posição 2

Questão 949

Dificuldade: Difícil
Usando busca binária adaptada, encontre um valor específico em um vetor ordenado que foi rotacionado em um ponto desconhecido.

Entrada: Vetor rotacionado [4, 5, 6, 7, 0, 1, 2], valor procurado = 0 **Saída:** Posição 4

Questão 950

Dificuldade: Difícil
Combine merge sort com contagem: durante a intercalação, conte o número de "inversões" (pares fora de ordem) presentes no vetor original.

Entrada: Vetor [8, 4, 2, 1] **Saída:** 6 inversões

Questão 951

Dificuldade: Difícil
Usando recursão, calcule a potência de um número (base elevada a expoente) de forma eficiente utilizando exponenciação rápida (dividir o expoente pela metade a cada chamada).

Entrada: Base = 3, expoente = 13 **Saída:** 1594323

Questão 952

Dificuldade: Difícil
Usando recursão pura (sem laços), some todos os elementos de um vetor de números inteiros.

Entrada: Vetor [4, 8, 15, 16, 23, 42] **Saída:** 108

Questão 953

Dificuldade: Difícil
Usando recursão, encontre o maior elemento de um vetor sem usar laços explícitos nem funções prontas de máximo.

Entrada: Vetor [3, 67, 12, 90, 5] **Saída:** 90

Questão 954

Dificuldade: Difícil
Implemente uma função recursiva que calcule todas as combinações de K elementos escolhidos de um conjunto de N elementos, sem repetição e sem considerar ordem.

Entrada: Conjunto [1, 2, 3, 4], K = 2 **Saída:** {1,2}, {1,3}, {1,4}, {2,3}, {2,4}, {3,4}

Questão 955

Dificuldade: Difícil
Usando backtracking, encontre todas as formas de dividir um conjunto de números em dois subconjuntos com somas iguais, informando se é possível.

Entrada: Conjunto [1, 5, 11, 5] **Saída:** Possível, subconjuntos {1, 5, 5} e {11}, ambos somam 11

Questão 956

Dificuldade: Difícil
Usando programação dinâmica, resolva o problema da subsequência crescente mais longa (LIS) em um vetor de números.

Entrada: Vetor [10, 9, 2, 5, 3, 7, 101, 18] **Saída:** Comprimento 4 (exemplo: 2, 3, 7, 18)

Questão 957

Dificuldade: Difícil
Usando programação dinâmica, calcule o número mínimo de operações (inserção, remoção, substituição) para transformar uma string em outra (distância de edição).

Entrada: String A = "gato", String B = "pato" **Saída:** 1 operação (substituir 'g' por 'p')

Questão 958

Dificuldade: Difícil
Usando programação dinâmica, calcule a distância de edição entre duas palavras mais distintas entre si.

Entrada: String A = "casa", String B = "caixa" **Saída:** 2 operações

Questão 959

Dificuldade: Difícil
Usando recursão com memoização, calcule o número de formas de decompor uma string de dígitos em números representando letras (A=1, B=2, ..., Z=26), sem contar zeros isolados como válidos.

Entrada: String "226" **Saída:** 3 formas (2-2-6, 22-6, 2-26)

Questão 960

Dificuldade: Difícil
Usando backtracking, encontre todas as palavras formadas ao percorrer células adjacentes (incluindo diagonais) em uma grade de letras, sem repetir a mesma célula (busca de palavras tipo "Boggle" simplificada), para uma palavra-alvo específica.

Entrada: Grade de letras 3x3, palavra-alvo "GATO" **Saída:** Verdadeiro, caminho encontrado percorrendo células adjacentes que formam "GATO"

Questão 961

Dificuldade: Difícil
Implemente DFS iterativo usando uma pilha explícita (sem recursão) para percorrer um grafo representado por lista de adjacência.

Entrada: Grafo: A-B, A-C, B-D; vértice inicial A **Saída:** Ordem de visita: A, C, B, D (ou equivalente dependendo da ordem de empilhamento)

Questão 962

Dificuldade: Difícil
Usando BFS, verifique se um grafo não direcionado é bipartido (pode ser colorido com apenas duas cores sem vértices adjacentes da mesma cor).

Entrada: Grafo: A-B, B-C, C-D, D-A **Saída:** Verdadeiro, é bipartido

Questão 963

Dificuldade: Difícil
Usando DFS, verifique se um grafo não direcionado é bipartido, dado um grafo que contém um ciclo ímpar.

Entrada: Grafo: A-B, B-C, C-A **Saída:** Falso, não é bipartido (ciclo de tamanho ímpar)

Questão 964

Dificuldade: Difícil
Implemente ordenação topológica de um grafo direcionado acíclico (DAG) usando DFS.

Entrada: Grafo direcionado: A->B, A->C, B->D, C->D **Saída:** Ordem topológica válida: A, C, B, D (ou A, B, C, D)

Questão 965

Dificuldade: Difícil
Usando DFS, determine se é possível concluir todas as N tarefas dado um conjunto de dependências entre elas (equivalente a detectar ciclo em grafo direcionado).

Entrada: 4 tarefas; dependências: tarefa 2 depende da 1, tarefa 3 depende da 2, tarefa 1 depende da 3
Saída: Falso, existe dependência cíclica

Questão 966

Dificuldade: Difícil
Usando busca binária em resposta (binary search on answer), encontre a menor capacidade de um caminhão necessária para transportar um conjunto de pacotes em no máximo D dias, respeitando a ordem dos pacotes.

Entrada: Pesos dos pacotes [1,2,3,4,5,6,7,8,9,10], dias disponíveis = 5 **Saída:** Capacidade mínima = 15

Questão 967

Dificuldade: Difícil
Usando recursão com backtracking, gere todas as sequências possíveis de lançamento de uma moeda (cara ou coroa) para N lançamentos, sem sequências consecutivas de duas caras.

Entrada: N = 3 lançamentos **Saída:** CCC seria inválida; válidas: CoCoCo, CoCoC, CoCCo, CCoCo, CCoC

Questão 968

Dificuldade: Difícil
Usando programação dinâmica, calcule o número máximo de valor obtido ao cortar uma haste de comprimento N em pedaços, dado uma tabela de preços por tamanho de pedaço (problema do corte de hastes).

Entrada: Comprimento = 4, preços por tamanho [1:1, 2:5, 3:8, 4:9] **Saída:** Valor máximo = 10 (cortando em dois pedaços de tamanho 2)

Questão 969

Dificuldade: Difícil
Usando programação dinâmica, calcule o número de maneiras distintas de multiplicar uma sequência de matrizes minimizando o número de multiplicações escalares (problema de cadeia de multiplicação de matrizes) — retorne apenas o custo mínimo.

Entrada: Dimensões das matrizes: 10x20, 20x30, 30x40 **Saída:** Custo mínimo = 18000

Questão 970

Dificuldade: Difícil
Usando recursão, determine todas as formas de combinar parênteses para calcular o valor máximo de uma expressão simples com operações de soma e multiplicação, alterando apenas a ordem de associação.

Entrada: Expressão "2+3*4" **Saída:** Valor máximo possível = 20 (agrupando como (2+3)*4)

Questão 971

Dificuldade: Difícil
Implemente busca binária para encontrar o número de ocorrências de um valor em um vetor ordenado, localizando a primeira e a última posição em que ele aparece.

Entrada: Vetor [2, 4, 4, 4, 4, 5, 7], valor = 4 **Saída:** Primeira posição 1, última posição 4, total de 4 ocorrências

Questão 972

Dificuldade: Difícil
Implemente contagem de inversões usando abordagem de divisão e conquista distinta do merge sort, aplicada a comparar quantos elementos à esquerda são maiores que elementos à direita.

Entrada: Vetor [5, 3, 2, 4, 1] **Saída:** 8 inversões

Questão 973

Dificuldade: Difícil
Usando recursão com backtracking, encontre o caminho de custo mínimo em uma grade de números, movendo-se apenas para a direita ou para baixo, somando os valores das células percorridas.

Entrada: Grade: [[1,3,1],[1,5,1],[4,2,1]] **Saída:** Caminho de custo mínimo = 7 (1→3→1→1→1)

Questão 974

Dificuldade: Difícil
Usando programação dinâmica, calcule o mesmo problema de caminho de custo mínimo em uma grade de números de forma eficiente (sem recomputações), retornando apenas o valor mínimo total.

Entrada: Grade: [[1,3,1],[1,5,1],[4,2,1]] **Saída:** Valor mínimo = 7

Questão 975

Dificuldade: Difícil
Usando backtracking, resolva o problema do caixeiro-viajante simplificado para 4 cidades, encontrando a rota de menor custo total que visita todas as cidades uma vez e retorna à origem.

Entrada: Matriz de distâncias entre 4 cidades (A, B, C, D) **Saída:** Rota de menor custo: A -> B -> D -> C -> A, custo total mínimo encontrado

Questão 976

Dificuldade: Difícil
Usando recursão, calcule o número de Catalan de ordem N, que representa, entre outras coisas, o número de árvores binárias distintas com N nós.

Entrada: N = 4 **Saída:** 14

Questão 977

Dificuldade: Difícil
Usando backtracking, gere todas as árvores binárias de busca estruturalmente distintas que podem ser formadas com valores de 1 a N (apenas conte quantas são possíveis).

Entrada: N = 3 **Saída:** 5 árvores distintas

Questão 978

Dificuldade: Difícil
Usando recursão, verifique se uma árvore binária é balanceada em altura (a diferença de altura entre subárvores esquerda e direita de qualquer nó não ultrapassa 1).

Entrada: Árvore binária com nó raiz 1, filho esquerdo 2 (com filho esquerdo 3), filho direito nulo **Saída:** Falso, não é balanceada

Questão 979

Dificuldade: Difícil
Usando recursão, calcule a altura de uma árvore binária representada por nós com referências para filho esquerdo e direito.

Entrada: Árvore com raiz A, filhos B e C; B tem filho D; D tem filho E **Saída:** Altura = 3

Questão 980

Dificuldade: Difícil
Usando busca em profundidade em árvore binária, verifique se existe um caminho da raiz até uma folha cuja soma dos valores seja igual a um valor alvo.

Entrada: Árvore com raiz 5, caminho 5-4-11-7, valor alvo = 27 **Saída:** Verdadeiro, caminho 5+4+11+7 = 27

Questão 981

Dificuldade: Difícil
Usando recursão com backtracking, gere todos os caminhos possíveis da raiz até as folhas de uma árvore binária, listando as somas de cada caminho.

Entrada: Árvore com raiz 1, filhos 2 e 3; 2 tem filhos 4 e 5 **Saída:** Caminhos: 1-2-4 (soma 7), 1-2-5 (soma 8), 1-3 (soma 4)

Questão 982

Dificuldade: Difícil
Usando recursão, converta um número decimal em sua representação binária, sem utilizar funções prontas de conversão de base.

Entrada: Número decimal = 45 **Saída:** "101101"

Questão 983

Dificuldade: Difícil
Usando recursão, converta uma string representando um número binário em seu valor decimal equivalente.

Entrada: String binária "110101" **Saída:** 53

Questão 984

Dificuldade: Difícil
Usando backtracking, resolva o problema do cavalo no tabuleiro de xadrez: determine se é possível visitar todas as casas de um tabuleiro 5x5 exatamente uma vez usando movimentos válidos de cavalo, a partir de uma casa inicial (passeio do cavalo).

Entrada: Tabuleiro 5x5, posição inicial (0,0) **Saída:** Verdadeiro, existe um passeio completo válido

Questão 985

Dificuldade: Difícil
Usando recursão com poda (branch and bound simplificado), encontre a soma máxima de um subconjunto de itens sem exceder um peso limite, interrompendo ramos que já ultrapassaram o limite antes de explorá-los completamente.

Entrada: Itens (peso): [4, 6, 3, 8], limite de peso = 10 **Saída:** Melhor combinação: [4, 6], soma de pesos = 10

Questão 986

Dificuldade: Difícil
Implemente o algoritmo de Dijkstra simplificado (sem heap, busca linear) para encontrar a menor distância entre dois vértices em um grafo ponderado com pesos positivos.

Entrada: Grafo ponderado: A-B(4), A-C(1), C-B(2), B-D(1), C-D(5); origem A, destino D **Saída:** Menor distância = 4 (caminho A-C-B-D)

Questão 987

Dificuldade: Difícil
Usando BFS modificado (0-1 BFS), encontre o menor custo entre dois vértices em um grafo cujas arestas têm peso 0 ou 1.

Entrada: Grafo: A-B(peso 0), B-C(peso 1), A-C(peso 1); origem A, destino C **Saída:** Custo mínimo = 1 (caminho A-B-C com custo 0+1, ou A-C direto com custo 1)

Questão 988

Dificuldade: Difícil
Usando recursão, calcule o número de subconjuntos de um conjunto de N elementos que somam um valor par, sem gerar explicitamente todos os subconjuntos (contagem via programação dinâmica).

Entrada: Conjunto [1, 2, 3], quantidade de subconjuntos com soma par **Saída:** 4 subconjuntos ({}, {2}, {1,3}, {1,2,3})

Questão 989

Dificuldade: Difícil
Usando recursão com memoização, calcule o número mínimo de quadrados perfeitos que somam exatamente um número dado.

Entrada: Número = 12 **Saída:** 3 quadrados (4 + 4 + 4)

Questão 990

Dificuldade: Difícil
Usando programação dinâmica, resolva o problema de particionamento de string em palíndromos, encontrando o número mínimo de cortes necessários para que todas as partes sejam palíndromos.

Entrada: String "aab" **Saída:** 1 corte (partições "aa" e "b")

Questão 991

Dificuldade: Difícil
Usando backtracking, gere todas as formas de particionar uma string em substrings que sejam todas palíndromos.

Entrada: String "aab" **Saída:** [{"a","a","b"}, {"aa","b"}]

Questão 992

Dificuldade: Difícil
Usando recursão com backtracking, resolva o problema de correspondência de padrões com curingas simples ('*' representa qualquer sequência, '?' representa um único caractere).

Entrada: Padrão "a*b?c", texto "aXXbYc" **Saída:** Verdadeiro, o texto corresponde ao padrão

Questão 993

Dificuldade: Difícil
Usando programação dinâmica, determine se uma string pode ser segmentada em uma sequência de palavras válidas presentes em um dicionário fornecido.

Entrada: String "gatobranco", dicionário ["gato", "branco", "gatobr"] **Saída:** Verdadeiro (segmentação "gato" + "branco")

Questão 994

Dificuldade: Difícil
Usando busca binária, encontre o elemento "pico" em um vetor (elemento maior que seus vizinhos imediatos), assumindo que o vetor cresce e depois decresce.

Entrada: Vetor [1, 3, 8, 12, 9, 5, 2] **Saída:** Elemento de pico = 12, na posição 3

Questão 995

Dificuldade: Difícil
Usando recursão com poda, resolva o problema de distribuir K itens idênticos entre N pessoas, determinando quantas distribuições distintas são possíveis onde cada pessoa recebe zero ou mais itens.

Entrada: K = 3 itens, N = 2 pessoas **Saída:** 4 distribuições possíveis: (0,3), (1,2), (2,1), (3,0)

Questão 996

Dificuldade: Difícil
Usando DFS em grafo representado por matriz de adjacência, conte o número de "ilhas" (grupos de células conectadas com valor 1) em uma grade binária, considerando conexões apenas horizontais e verticais.

Entrada: Grade: [[1,1,0,0],[1,0,0,1],[0,0,1,1],[0,0,0,0]] **Saída:** 3 ilhas

Questão 997

Dificuldade: Difícil
Usando BFS em grade binária, calcule a menor distância (em passos) entre duas células livres, considerando que células com valor 1 são obstáculos intransponíveis.

Entrada: Grade 4x4 com obstáculos, origem (0,0), destino (3,3) **Saída:** Menor distância = 6 passos

Questão 998

Dificuldade: Difícil
Usando recursão com memoização, resolva o problema do troco de moedas contando não apenas o número mínimo de moedas, mas também retornando quais moedas compõem essa solução mínima.

Entrada: Moedas disponíveis [1, 5, 6, 8], valor alvo = 11 **Saída:** Mínimo de 2 moedas (5 + 6), não 3 (o resultado guloso 8+1+1+1 seria pior)

Questão 999

Dificuldade: Difícil
Usando algoritmo de Kahn (BFS) para ordenação topológica, determine a ordem de execução de tarefas dado um grafo de dependências direcionado, ou informe se não é possível devido a um ciclo.

Entrada: Tarefas A, B, C, D; dependências: A antes de B, A antes de C, B antes de D, C antes de D
Saída: Ordem válida: A, B, C, D (ou A, C, B, D)

Questão 1000

Dificuldade: Difícil
Usando recursão com backtracking e poda por limite superior, resolva o problema da soma de subconjuntos mais próxima possível de um valor alvo sem ultrapassá-lo, retornando a soma máxima encontrada.

Entrada: Conjunto [3, 7, 9, 14, 5], alvo máximo = 20 **Saída:** Soma máxima sem ultrapassar = 19 (subconjunto 5 + 14, ou 3+7+9)